Deep Learning · Day 26 of 30 · about 60 minutes

Overfitting, regularisation and real-world frameworks

🎯 Today's goal

By the end of this hour you will:

- spot **overfitting** and **underfitting** from the gap between training and test scores
- use a **validation set** to choose settings, and keep the test set for the very end
- see why **more data** helps, with a learning curve
- fight overfitting with **regularisation**: `max_depth`, `alpha`, `early_stopping` and dropout
- read real **PyTorch** and **Keras** code and match it to what you built by hand

🤔 Why this matters

Imagine two students preparing for an exam with last year's questions. One **memorises** the answers word for word: perfect on last year's paper, lost when the questions change. The other learns the **ideas**: a little less perfect on the practice paper, but fine on the real exam. Models can be either student.

Yesterday your network scored almost 100% on its training pictures and a bit less on new ones. That difference is the most important warning sign in machine learning, and big neural networks, with their thousands or millions of weights, are especially good at memorising. Today you'll learn to see it and to fix it.

🌙 A noisy, made-up dataset

scikit-learn's `make_moons` invents points shaped like two interlocking half-moons, two classes (0 and 1). With `noise=0.35` the points are shaken up, so some land on the "wrong" side. No model can be perfect on new points here, which makes it ideal for spotting memorising.

We split the 400 made-up points **three** ways:

Set	Size	Used for	Exam analogy
training	240	learning the weights (`.fit`)	the textbook
validation	80	comparing models and settings	practice exams
test	80	one final, honest score at the very end	the real exam

```
import numpy as np
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split

X, y = make_moons(n_samples=400, noise=0.35, random_state=42)

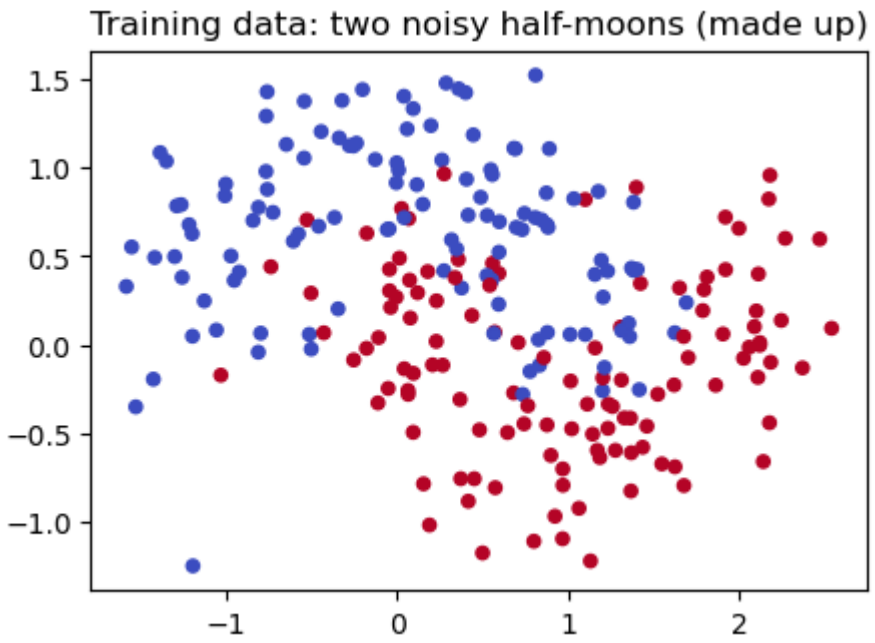
# First keep 20% aside as the test set,
# then split the rest into training and validation
X_rest, X_test, y_rest, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
X_train, X_val, y_train, y_val = train_test_split(
    X_rest, y_rest, test_size=0.25, random_state=42, stratify=y_rest)

print("training:  ", X_train.shape)
print("validation:", X_val.shape)
print("test:      ", X_test.shape)
```

```
training:   (240, 2)
validation: (80, 2)
test:       (80, 2)
```

```
import matplotlib.pyplot as plt

plt.figure(figsize=(5, 3.5))
plt.scatter(X_train[:, 0], X_train[:, 1], c=y_train, cmap="coolwarm", s=20)
plt.title("Training data: two noisy half-moons (made up)")
plt.show()
```



🎯 Underfitting, overfitting, and the sweet spot

Decision trees (Day 19) make this easy to see, because `max_depth` directly controls how complicated the tree may get. Compare a tiny tree, a medium tree and a tree with no limit:

```
from sklearn.tree import DecisionTreeClassifier

for depth in [1, 3, None]:          # None = no limit
    tree = DecisionTreeClassifier(max_depth=depth, random_state=42)
    tree.fit(X_train, y_train)
    train_acc = tree.score(X_train, y_train)
    val_acc = tree.score(X_val, y_val)
    gap = train_acc - val_acc
    print(f"max_depth={str(depth):4}  train {train_acc:.3f}  validation {val_acc:.3f}
gap {gap:+.3f}")
```

```
max_depth=1     train 0.762  validation 0.738  gap +0.025
max_depth=3     train 0.838  validation 0.850  gap -0.012
max_depth=None  train 1.000  validation 0.838  gap +0.162
```

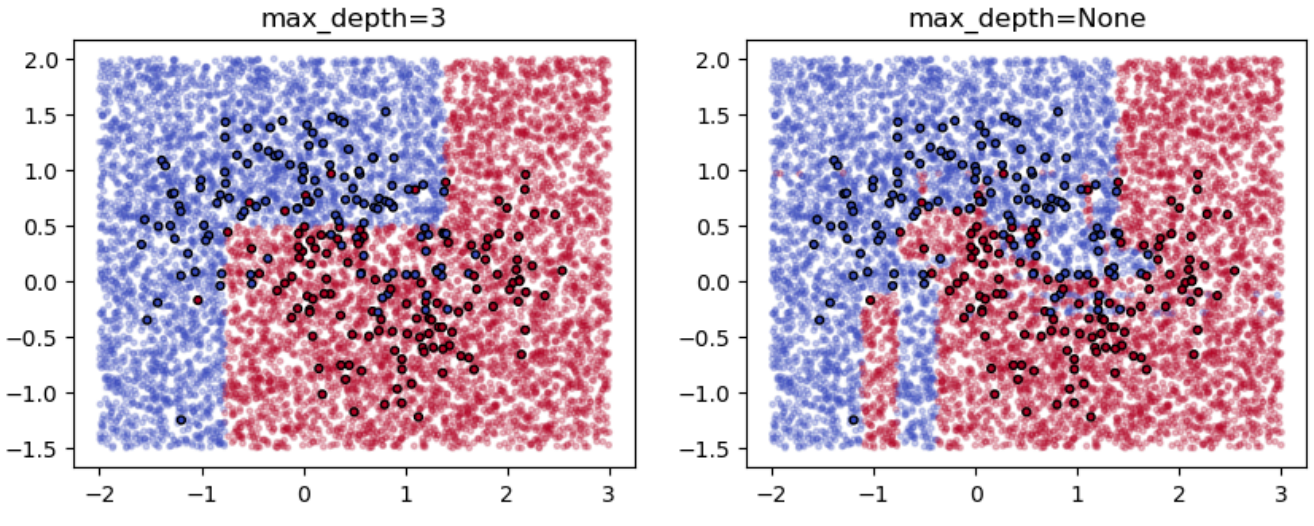
- **Underfitting** (depth 1): bad on training *and* validation. The model is too simple to capture the pattern, like a student who didn't study.

- **Overfitting** (no limit): perfect on training, clearly worse on validation. A big **gap** means it memorised the training points, noise included.
- **Good fit** (depth 3): similar scores on both, and one of the best validation scores.

Pictures make this clearer. Let's colour the whole plane by what each tree predicts, using 8000 random points:

```
rng = np.random.default_rng(42)
grid = rng.uniform([-2, -1.5], [3, 2], size=(8000, 2))   # random points

fig, axes = plt.subplots(1, 2, figsize=(10, 3.5))
for ax, depth in zip(axes, [3, None]):
    tree = DecisionTreeClassifier(max_depth=depth, random_state=42)
    tree.fit(X_train, y_train)
    # background: the tree's answer for every random point
    ax.scatter(grid[:, 0], grid[:, 1], c=tree.predict(grid),
               cmap="coolwarm", s=6, alpha=0.25)
    # the training points, with a black edge
    ax.scatter(X_train[:, 0], X_train[:, 1], c=y_train,
               cmap="coolwarm", s=12, edgecolors="black")
    ax.set_title(f"max_depth={depth}")
plt.show()
```

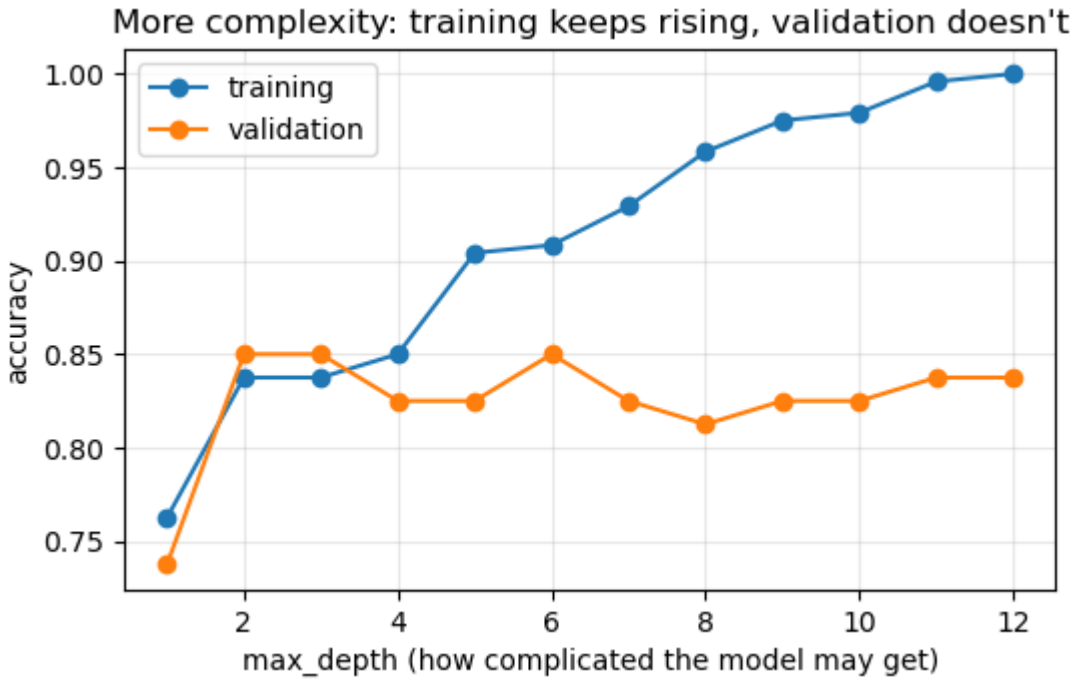


The depth-3 tree draws a few simple regions. The unlimited tree draws tiny islands around single noisy points: it has learned the noise, and new points that land in those islands get the wrong answer.

Here is the full picture, for depths 1 to 12:

```
depths = range(1, 13)
train_scores = []
val_scores = []
for depth in depths:
    tree = DecisionTreeClassifier(max_depth=depth, random_state=42)
    tree.fit(X_train, y_train)
    train_scores.append(tree.score(X_train, y_train))
    val_scores.append(tree.score(X_val, y_val))

plt.figure(figsize=(6, 3.5))
plt.plot(depths, train_scores, marker="o", label="training")
plt.plot(depths, val_scores, marker="o", label="validation")
plt.xlabel("max_depth (how complicated the model may get)")
plt.ylabel("accuracy")
plt.title("More complexity: training keeps rising, validation doesn't")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```



🧪 Why a validation set?

We just chose a depth by looking at scores. If we had used the **test** set for that, we would have picked the depth that happens to suit those 80 test points, and the test score would no longer be an honest "unseen data" score. It's like practising with the real exam paper. So: **choose with the validation set, then use the test set once**, at the very end.

```
# the depth with the best validation score
best_depth = depths[val_scores.index(max(val_scores))]
print("best depth by validation:", best_depth)

final_tree = DecisionTreeClassifier(max_depth=best_depth, random_state=42)
final_tree.fit(X_train, y_train)
print(f"final test accuracy: {final_tree.score(X_test, y_test):.3f}")
```

```
best depth by validation: 2
final test accuracy: 0.900
```

In a hurry, it's easy to create a model and ask it for predictions before training it. scikit-learn stops you:

💡 This example has a mistake on purpose. Run it and read the error.

```
new_tree = DecisionTreeClassifier(max_depth=3)
new_tree.predict(X_test)      # oops: no .fit() yet
```

⚠️ **NotFittedError**

```
Traceback (most recent call last):
  line 2, in <module>
    new_tree.predict(X_test)      # oops: no .fit() yet
    ~~~~~~~~~~~~~~~~^^^^^^^^
sklearn.exceptions.NotFittedError: This DecisionTreeClassifier instance is not
fitted yet. Call 'fit' with appropriate arguments before using this estimator.
```

A model that hasn't seen any data knows nothing. Call `.fit(X_train, y_train)` first.

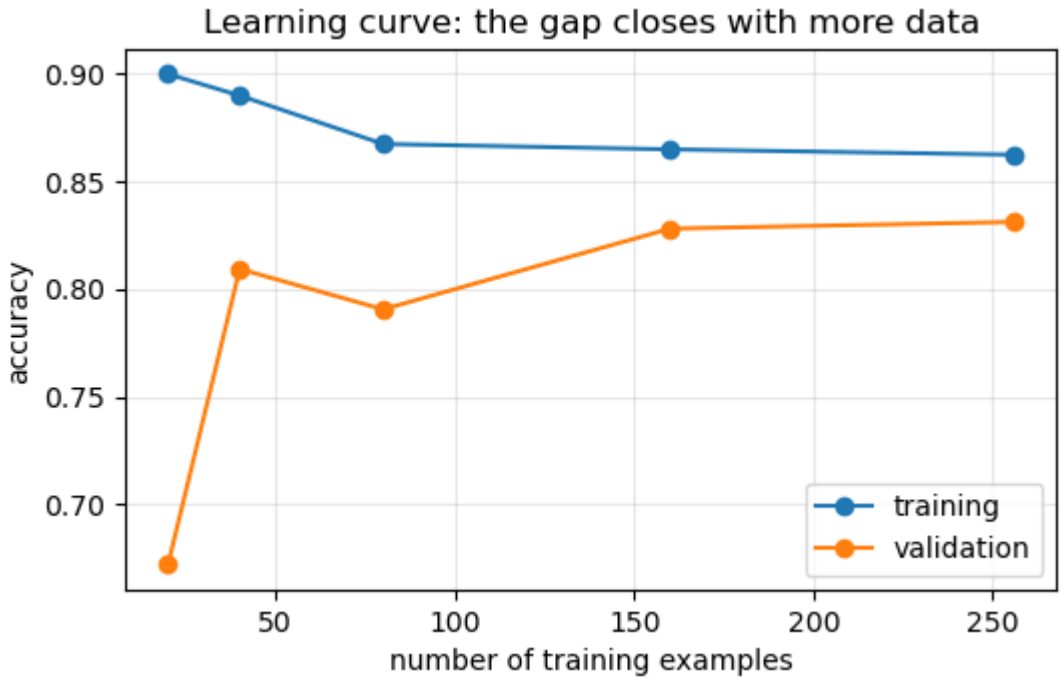
📚 More data helps

The best cure for overfitting is more examples: it's hard to memorise 10 000 points, so the model is pushed to learn the real pattern. A **learning curve** shows the scores as the training set grows. `learning_curve` does this with cross-validation (Day 20) and gives one score per fold; we take the mean of each row:

```
from sklearn.model_selection import learning_curve

sizes, train_lc, val_lc = learning_curve(
    DecisionTreeClassifier(max_depth=3, random_state=42), X_rest, y_rest,
    train_sizes=[20, 40, 80, 160, 256], cv=5, shuffle=True, random_state=42)

plt.figure(figsize=(6, 3.5))
plt.plot(sizes, train_lc.mean(axis=1), marker="o", label="training")
plt.plot(sizes, val_lc.mean(axis=1), marker="o", label="validation")
plt.xlabel("number of training examples")
plt.ylabel("accuracy")
plt.title("Learning curve: the gap closes with more data")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```



With 20 examples the model looks great on training data and poor on new data. With more examples the two lines come close together. In deep learning, collecting more (and cleaner) data is often worth more than any clever trick.

🧲 Regularisation: keeping a network modest

When you can't get more data, you **regularise**: you hold the model back from becoming too complicated. For a tree that's `max_depth`. For a neural network there are three common tools.

1. Weight penalty: `alpha`

Overfitting networks tend to grow some very large weights to twist themselves around single points. `alpha` adds a penalty for large weights to the loss (called **L2 regularisation**), so the network prefers small, calm weights and smoother answers. Bigger `alpha` = stronger penalty. Here a big network (two hidden layers of 64) gets only **60** training points, a recipe for memorising:

```
import warnings
from sklearn.exceptions import ConvergenceWarning
from sklearn.neural_network import MLPClassifier

warnings.filterwarnings("ignore", category=ConvergenceWarning)

X_small, y_small = X_train[:60], y_train[:60]

for alpha in [0.0001, 1.0]:
    net = MLPClassifier(hidden_layer_sizes=(64, 64), alpha=alpha,
                        learning_rate_init=0.01, max_iter=500, random_state=42)
    net.fit(X_small, y_small)
    train_acc = net.score(X_small, y_small)
    val_acc = net.score(X_val, y_val)
    print(f"alpha={alpha:<7} train {train_acc:.3f}   validation {val_acc:.3f}")
```

```
alpha=0.0001  train 1.000   validation 0.812
alpha=1.0     train 0.817   validation 0.863
```

With almost no penalty, the network is perfect on its 60 points and worse on new ones. With `alpha=1.0` it is less perfect on training data but better on validation: exactly the trade we want. (`alpha` is a setting you choose by validation, like `max_depth`.)

2. Early stopping

During training, a network first learns the general pattern and later starts memorising. **Early stopping** watches a validation score while training and stops when it hasn't improved for a while. With `early_stopping=True`, `MLPClassifier` keeps 10% of the training data aside for this and stops after 10 epochs without improvement. `n_iter_` tells you how many epochs it really ran:

```
for stop_early in [False, True]:
    net = MLPClassifier(hidden_layer_sizes=(64, 64), early_stopping=stop_early,
                        learning_rate_init=0.01, max_iter=300, random_state=42)
    net.fit(X_train, y_train)
    val_acc = net.score(X_val, y_val)
    print(f"early_stopping={str(stop_early):5}  epochs: {net.n_iter_:3}   validation
{val_acc:.3f}")
```

```
early_stopping=False  epochs: 115   validation 0.787
early_stopping=True   epochs:  28   validation 0.863
```

Fewer epochs, less time, and here even a better validation score.

3. Dropout (explained, not run)

Dropout is the favourite trick in big networks. During training, every step randomly switches off a share of the neurons (for example 20%). The network can never rely on one particular neuron, so knowledge gets spread out, like a team where anyone might be absent on a given day, so everybody learns the whole job. When the network is used for predictions, all neurons are switched back on. scikit-learn's `MLPClassifier` doesn't have dropout, but the frameworks below do: it's a single line.

🏭 Real-world frameworks: PyTorch and Keras

Professionals build networks with **PyTorch** (from Meta) or **Keras** (on top of Google's TensorFlow). They can't run on this page (they are far too big for the browser), but you can already read them. Both examples below build the digits network from yesterday: 64 inputs → 64 hidden neurons → 10 outputs.

PyTorch · *read-only example, not runnable here*

```
import torch
from torch import nn

# 64 inputs → 64 hidden neurons (ReLU) → 10 outputs
model = nn.Sequential(
    nn.Linear(64, 64),      # weights + biases, like W1 and b1
    nn.ReLU(),              # the activation
    nn.Dropout(0.2),        # dropout: switch off 20% while training
    nn.Linear(64, 10),      # like W2 and b2
)
loss_fn = nn.CrossEntropyLoss()      # log-loss for classes
optimizer = torch.optim.Adam(model.parameters(), lr=0.001,
                             weight_decay=0.0001)   # like alpha

X = torch.tensor(X_train, dtype=torch.float32)
y = torch.tensor(y_train)

for epoch in range(100):             # 100 epochs
    prediction = model(X)            # forward pass
    loss = loss_fn(prediction, y)    # how wrong?
    optimizer.zero_grad()            # clear the old gradients
    loss.backward()                  # backpropagation, done for you
    optimizer.step()                 # one gradient-descent step
```

That's yesterday's loop: forward pass, loss, backpropagation, update. The difference is that `loss.backward()` works out every gradient automatically, for any network you can imagine. Keras hides the loop completely:

Keras · *read-only example, not runnable here*

```
from tensorflow import keras

model = keras.Sequential([
    keras.Input(shape=(64,)),
    keras.layers.Dense(64, activation="relu"),
    keras.layers.Dropout(0.2),
    keras.layers.Dense(10, activation="softmax"),
])
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])

stop = keras.callbacks.EarlyStopping(patience=10)   # early stopping
model.fit(X_train, y_train, epochs=200,
          validation_split=0.1, callbacks=[stop])
model.evaluate(X_test, y_test)
```

Every piece has a name you now know:

Idea	By hand (Day 24)	scikit-learn	PyTorch / Keras
layers	`W1`, `b1`, `W2`, `b2`	`hidden_layer_sizes`	`nn.Linear` / `Dense`
activation	`sigmoid`	`activation="relu"`	`nn.ReLU` / `activation="relu"`
loss	MSE	log-loss (built in)	`CrossEntropyLoss` / `loss=…`
gradients	backprop written by hand	automatic	`loss.backward()` / automatic
optimiser	`W -= lr * grad`	`solver="adam"`	`Adam` / `optimizer="adam"`
epochs	`for epoch in range(…)`	`max_iter`	your loop / `epochs=`
regularisation	none	`alpha`, `early_stopping`	`weight_decay`, `Dropout`, `EarlyStopping`

Why GPUs? A network's work is mostly matrix products like `X @ W1`: millions of multiply-and-add steps that don't depend on each other. A **GPU** (graphics card) was built to colour millions of pixels at once, so it can do thousands of these multiplications at the same time. That's why big models train on GPUs, and why PyTorch has one line to move a model there: `model.to("cuda")`. Free online notebooks such as Google Colab let you try a GPU without owning one.

⚠️ Common mistakes

- Only looking at the training score. Always compare it with validation or test: the gap is what matters.
- Choosing settings with the test set. Then the test score is no longer honest. Use a validation set.
- Thinking "bigger network = better". Without enough data, a bigger network mostly memorises more.
- Regularising too hard (huge `alpha`, tiny `max_depth`): now the model underfits.
- Calling `.predict` before `.fit` → `NotFittedError`.

✅ Recap

- **Underfitting**: bad on training and new data. **Overfitting**: great on training, worse on new data (a big gap).
- Three sets: **train** to learn, **validation** to choose, **test** once at the end.
- A **learning curve** shows that more data closes the gap.
- **Regularisation** holds a model back: `max_depth` (trees), `alpha` (weight penalty), `early_stopping=True`, dropout.
- PyTorch and Keras use the same pieces you built: layers, activation, loss, backprop, optimiser, epochs.

🏋️ Exercises

Write your answer in the space under each exercise. Try it on the computer too, by opening this day's `index.html`. Solutions are in the answer key at the end.

1. Measure the gap

Train a decision tree with **no depth limit** (`random_state=42`) on the training set. Store its training accuracy in `train_acc`, its test accuracy in `test_acc`, and the difference `train_acc - test_acc` in `gap`.

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

X, y = make_moons(n_samples=400, noise=0.35, random_state=42)
X_rest, X_test, y_rest, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
X_train, X_val, y_train, y_val = train_test_split(
    X_rest, y_rest, test_size=0.25, random_state=42, stratify=y_rest)

# Train the tree, then store train_acc, test_acc and gap
```

Hints:

1. `tree = DecisionTreeClassifier(random_state=42)` has no depth limit. Then `tree.fit(X_train, y_train)`.
2. `train_acc = tree.score(X_train, y_train)`, and the same with `X_test, y_test`.
3. `gap = train_acc - test_acc`

2. Find the best depth

Try `max_depth` from 1 to 10 (with `random_state=42`). For each, train on the training set and measure accuracy on the **validation** set. Store the depth with the highest validation accuracy in `best_depth` (if several tie, the smallest one).

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

X, y = make_moons(n_samples=400, noise=0.35, random_state=42)
X_rest, X_test, y_rest, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
X_train, X_val, y_train, y_val = train_test_split(
    X_rest, y_rest, test_size=0.25, random_state=42, stratify=y_rest)

best_depth = None
best_score = 0

# Loop over the depths 1 to 10
```

Hints:

1. `for depth in range(1, 11):` goes through 1, 2, ..., 10.
2. Inside: train a tree with `max_depth=depth`, then `score = tree.score(X_val, y_val)`.
3. `if score > best_score:` then `best_score = score` and `best_depth = depth`. Using `>` (not `>=`) keeps the smallest depth when there is a tie.

3. Choose alpha by validation

For each `alpha` in the list, train the given network on the training set and store its **validation** accuracy in the dictionary `val_scores` (key = alpha). Then store the alpha with the highest validation accuracy in `best_alpha`.

```
import warnings
from sklearn.exceptions import ConvergenceWarning
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier

warnings.filterwarnings("ignore", category=ConvergenceWarning)

X, y = make_moons(n_samples=400, noise=0.35, random_state=42)
X_rest, X_test, y_rest, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
X_train, X_val, y_train, y_val = train_test_split(
    X_rest, y_rest, test_size=0.25, random_state=42, stratify=y_rest)

alphas = [0.0001, 0.1, 1.0]
val_scores = {}

for alpha in alphas:
    net = MLPClassifier(hidden_layer_sizes=(64, 64), alpha=alpha,
                        learning_rate_init=0.01, max_iter=300, random_state=42)
    # Train net and store its validation accuracy in val_scores[alpha]

# Store best_alpha
```

Hints:

1. Inside the loop: `net.fit(X_train, y_train)`, then `val_scores[alpha] = net.score(X_val, y_val)`.
2. After the loop, find the key with the biggest value. A loop over `val_scores.items()` with an `if` works.
3. A short way: `best_alpha = max(val_scores, key=val_scores.get)` (like `sorted(..., key=len)` from Day 6).

4. Stop early

Create an `MLPClassifier` called `model` with `hidden_layer_sizes=(64, 64)`, `learning_rate_init=0.01`, `max_iter=300`, `random_state=42` and **early stopping switched on**. Train it, then store the number of epochs it actually ran in `n_epochs`.

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier

X, y = make_moons(n_samples=400, noise=0.35, random_state=42)
X_rest, X_test, y_rest, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
X_train, X_val, y_train, y_val = train_test_split(
    X_rest, y_rest, test_size=0.25, random_state=42, stratify=y_rest)

# Create model, train it, store n_epochs
```

Hints:

1. Early stopping is switched on with `early_stopping=True` inside `MLPClassifier(...)`.
2. Train with `model.fit(X_train, y_train)`.
3. The number of epochs it ran is `model.n_iter_`.

5. Challenge: pick a model the honest way

Three candidate models are ready in the dictionary `candidates`. Do it like a professional:

- train each one on the training set and store its validation accuracy in `val_scores` (key = the model's name)
- store the name of the model with the best validation accuracy in `best_name`
- only then, store **that** model's test accuracy in `test_score`

```
import warnings
from sklearn.exceptions import ConvergenceWarning
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.neural_network import MLPClassifier

warnings.filterwarnings("ignore", category=ConvergenceWarning)

X, y = make_moons(n_samples=400, noise=0.35, random_state=42)
X_rest, X_test, y_rest, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
X_train, X_val, y_train, y_val = train_test_split(
    X_rest, y_rest, test_size=0.25, random_state=42, stratify=y_rest)

candidates = {
    "shallow tree": DecisionTreeClassifier(max_depth=3, random_state=42),
    "deep tree": DecisionTreeClassifier(random_state=42),
    "network with alpha=1": MLPClassifier(
        hidden_layer_sizes=(64, 64), alpha=1.0,
        learning_rate_init=0.01, max_iter=300, random_state=42),
}
val_scores = {}

# Train and score every candidate, then pick best_name and measure test_score
```

Hints:

1. `for name, model in candidates.items():` then `model.fit(X_train, y_train)` and `val_scores[name] = model.score(X_val, y_val)`.
2. `best_name = max(val_scores, key=val_scores.get)` finds the name with the biggest score.
3. The models were trained inside the loop, so: `test_score = candidates[best_name].score(X_test, y_test)`.

➡️ Tomorrow

Text basics: cleaning and tokenising. A new phase begins: natural language processing (NLP). So far every input was already a number. Tomorrow you'll take messy made-up product reviews, clean them up (lowercase, no punctuation, no filler words) and split them into words, the first step towards models that understand text.

🔑 Answer key

Try the exercises yourself first! There is often more than one correct answer.

1. Measure the gap

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

X, y = make_moons(n_samples=400, noise=0.35, random_state=42)
X_rest, X_test, y_rest, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
X_train, X_val, y_train, y_val = train_test_split(
    X_rest, y_rest, test_size=0.25, random_state=42, stratify=y_rest)

tree = DecisionTreeClassifier(random_state=42)
tree.fit(X_train, y_train)
train_acc = tree.score(X_train, y_train)
test_acc = tree.score(X_test, y_test)
gap = train_acc - test_acc
print(f"train {train_acc:.3f}, test {test_acc:.3f}, gap {gap:.3f}")
```

2. Find the best depth

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

X, y = make_moons(n_samples=400, noise=0.35, random_state=42)
X_rest, X_test, y_rest, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
X_train, X_val, y_train, y_val = train_test_split(
    X_rest, y_rest, test_size=0.25, random_state=42, stratify=y_rest)

best_depth = None
best_score = 0

for depth in range(1, 11):
    tree = DecisionTreeClassifier(max_depth=depth, random_state=42)
    tree.fit(X_train, y_train)
    score = tree.score(X_val, y_val)
    if score > best_score:
        best_score = score
        best_depth = depth
print(f"best depth: {best_depth} (validation {best_score:.3f})")
```

3. Choose alpha by validation

```
import warnings
from sklearn.exceptions import ConvergenceWarning
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier

warnings.filterwarnings("ignore", category=ConvergenceWarning)

X, y = make_moons(n_samples=400, noise=0.35, random_state=42)
X_rest, X_test, y_rest, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
X_train, X_val, y_train, y_val = train_test_split(
    X_rest, y_rest, test_size=0.25, random_state=42, stratify=y_rest)

alphas = [0.0001, 0.1, 1.0]
val_scores = {}

for alpha in alphas:
    net = MLPClassifier(hidden_layer_sizes=(64, 64), alpha=alpha,
                        learning_rate_init=0.01, max_iter=300, random_state=42)
    net.fit(X_train, y_train)
    val_scores[alpha] = net.score(X_val, y_val)

best_alpha = max(val_scores, key=val_scores.get)
print(val_scores)
print("best alpha:", best_alpha)
```

4. Stop early

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier

X, y = make_moons(n_samples=400, noise=0.35, random_state=42)
X_rest, X_test, y_rest, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
X_train, X_val, y_train, y_val = train_test_split(
    X_rest, y_rest, test_size=0.25, random_state=42, stratify=y_rest)

model = MLPClassifier(hidden_layer_sizes=(64, 64), learning_rate_init=0.01,
                      max_iter=300, random_state=42, early_stopping=True)
model.fit(X_train, y_train)
n_epochs = model.n_iter_
print(f"stopped after {n_epochs} epochs instead of {model.max_iter}")
```

5. Challenge: pick a model the honest way

```
import warnings
from sklearn.exceptions import ConvergenceWarning
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.neural_network import MLPClassifier

warnings.filterwarnings("ignore", category=ConvergenceWarning)

X, y = make_moons(n_samples=400, noise=0.35, random_state=42)
X_rest, X_test, y_rest, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
X_train, X_val, y_train, y_val = train_test_split(
    X_rest, y_rest, test_size=0.25, random_state=42, stratify=y_rest)

candidates = {
    "shallow tree": DecisionTreeClassifier(max_depth=3, random_state=42),
    "deep tree": DecisionTreeClassifier(random_state=42),
    "network with alpha=1": MLPClassifier(
        hidden_layer_sizes=(64, 64), alpha=1.0,
        learning_rate_init=0.01, max_iter=300, random_state=42),
}
val_scores = {}

for name, model in candidates.items():
    model.fit(X_train, y_train)
    val_scores[name] = model.score(X_val, y_val)

best_name = max(val_scores, key=val_scores.get)
test_score = candidates[best_name].score(X_test, y_test)
print(val_scores)
print(f"chosen: {best_name}, test accuracy {test_score:.3f}")
```